

● ★ Firmware Engineer – Tell Me About Yourself (Ovarro-Ready Version)

- I am an Embedded Firmware Engineer with strong experience developing low-power, network-connected devices using C and C++. Over the last few years, I've worked extensively with microcontrollers such as STM32, ESP32, and ARM Cortex-M, building firmware that integrates sensors, communication stacks, and real-time control.
- A lot of my recent work has focused on IoT systems — implementing Wi-Fi, BLE, MQTT, and secure communication using TLS/HTTPS, as well as designing efficient power-management, sleep modes, and OTA update mechanisms. I also have hands-on experience writing device drivers for peripherals like UART, SPI, I2C, ADC, PWM, and handling interrupts and RTOS-based scheduling.
- I enjoy working on products that operate in the field for long periods, where reliability, security, and power efficiency are critical — which is why the role at Ovarro really interests me.
- Your work on battery-powered dataloggers, LTE-M/NB-IoT connectivity, and secure data transmission aligns very closely with my background and the type of engineering I want to continue developing.

★ Why Ovarro?

I am really interested in Ovarro because the company works on technology that has a real impact on critical infrastructure — especially in water, utilities, and monitoring systems. I enjoy working on embedded devices that need to be reliable, low-power, and secure in the field, and that aligns very closely with the products Ovarro develops.

I'm also drawn to the mix of IoT connectivity, sensor integration, and long-life battery-powered devices. That's exactly the type of engineering I've been focusing on, so it feels like a natural next step for me.

And finally, Ovarro's reputation for innovation and long-term product reliability makes it a place where I feel I can contribute and continue growing as a firmware engineer.

★ Embedded Background Summary (Ovarro-Ready Version)

My background is in embedded firmware development, mainly using C and C++ on microcontrollers such as STM32, ESP32, and ARM Cortex-M devices. I have worked on IoT-focused systems involving Wi-Fi, BLE, MQTT, and secure communication using TLS/HTTPS, as well as low-power design, sleep modes, and battery-optimised firmware.

I have hands-on experience writing device drivers for peripherals like UART, SPI, I2C, ADC, PWM, and handling interrupts, timers, and RTOS-based scheduling. A lot of my work has involved integrating sensors, managing data logging, and ensuring reliable communication in field-deployed devices.

Overall, my experience is centred around building secure, reliable, low-power embedded systems — which aligns very closely with the type of products Ovarro develops.

Questions at the end

- What is the expected hiring timeline for this role?
- What projects or products will this firmware engineer be working on?
- What does the first 3–6 months look like in this position?

★ Firmware Interview Closing Statement (Perfect for Ovarro)

Thanks, Greg, I have really enjoyed our conversation today. The role sounds like a great match for my experience in embedded firmware, especially around low-power IoT devices, secure communication, and microcontroller development. I am definitely interested in progressing to the next stage, and I'd be happy to review the job description once it's available. Please let me know if you need anything else from me at this point.

★ How to explain Secure IOT this in your interview (perfect answer)

In my ESP32 firmware, I implemented secure MQTT using `WiFiClientSecure`. When TLS is enabled, the MQTT client switches from a plain TCP socket to a TLS-encrypted socket using `_mqtt.setClient(_secureClient)`. All telemetry and command messages are then transmitted over an encrypted TLS channel, ensuring secure IoT communication.”

★ How to explain this in your interview (perfect answer)

At Park Signalling, I worked directly with cellular modems (GSM-R) using AT commands over UART. We went to Network Rail’s GSM-R reference lab and performed live tests to verify that our TrioRail modem could register on the GSM-R network, set up a PDP context, and send/receive TCP and UDP data to our server. We also checked SIM ICCID, signal strength, and the assigned IP address. We used Terminal YAT to send AT commands and read the modem’s responses in real time. LTE-M and NB-IoT modems work in exactly the same way — they’re controlled over UART using AT commands for network registration, PDP setup, and data transmission. So my experience transfers directly to LTE-M/NB-IoT.

★ How to combine everything into one powerful statement

- 1) My experience fits very well with Ovarro’s products because I’ve worked on low-power embedded systems that collect sensor data, process it efficiently, and transmit it securely over IoT protocols. I’ve built firmware for MCUs like STM32 and ESP32, written drivers for ADC, I2C, SPI, and UART, and implemented secure communication using TLS/HTTPS and MQTT. I’ve also designed low-power sampling loops, sleep modes, and OTA update mechanisms — all of which are core to Ovarro’s dataloggers and remote monitoring devices.

- 2) In addition, I have hands-on experience working with cellular modems using AT commands: at Park Signalling I tested GSM-R modems on Network Rail's reference lab, registering on the network, setting PDP context, checking SIM and signal, and sending/receiving TCP/UDP data over a live cellular link. LTE-M and NB-IoT modems work in exactly the same way, so my cellular and secure IoT experience transfers directly to Ovarro's communication stack.